



MINERÍA DE DATOS Y MODELIZACIÓN PREDICTIVA

Análisis Clustering

Daniel Martín García

1. Introducción al Análisis de Clustering

En el vasto panorama de la recopilación de datos, la inclusión de un extenso conjunto de variables es una práctica común. Sin embargo, la abundancia de variables puede generar desafíos significativos. En este contexto de complejidad multidimensional, surge la necesidad de técnicas que simplifiquen y resuman eficientemente la información. Aquí es donde el Análisis de Clustering se erige como una herramienta crucial.

Paralelamente, en el ámbito del machine learning, los algoritmos supervisados dominan el escenario. Estos algoritmos se basan en datos previamente etiquetados, lo que implica un aprendizaje 'supervisado'. Sin embargo, existe una línea alternativa de métodos clasificatorios que operan sin etiquetas predefinidas, centrándose en la creación de agrupaciones a partir de los datos en lugar de clasificarlos en categorías fijas. Esta aproximación, que forma parte esencial de la Ciencia de los Datos, es crucial cuando se desconocen características distintivas entre los grupos.

El Clustering, o análisis de agrupamiento, es una técnica estadística que busca organizar datos en grupos coherentes, donde elementos similares entre sí compartan un mismo grupo. En un entorno donde los datos masivos predominan, la capacidad de identificar patrones de manera efectiva sin perder características fundamentales se convierte en un recurso poderoso.

Uno de los desafíos primordiales que el Clustering aborda es la identificación de estructuras y patrones dentro de conjuntos de datos. En situaciones donde la clasificación manual es impracticable, el Clustering permite la agrupación automática de observaciones relacionadas, brindando una visión más clara de las relaciones intrínsecas en los datos.

Además, el Clustering facilita la interpretación visual de los datos. Al reducir la complejidad a través de la formación de grupos, es posible representar gráficamente relaciones en un espacio de baja dimensión. Esta visualización es inestimable para comprender y descubrir patrones en conjuntos de datos complejos.

La versatilidad del Clustering es evidente en su aplicabilidad a diversas disciplinas, incluyendo biología, economía y psicología, por mencionar algunas. Su capacidad para identificar patrones ocultos ha llevado a avances notables en campos tan variados como la genómica y las finanzas.

El Clustering no solo contribuye a la interpretación de datos, sino que también mejora la calidad del análisis al eliminar variables redundantes. Este proceso de simplificación no solo aumenta la precisión de los modelos sino que también optimiza los recursos computacionales, facilitando el manejo y procesamiento de grandes conjuntos de datos.

En conclusión, el Análisis de Clustering va más allá de ser una simple técnica de agrupamiento. Es una herramienta multifacética que aborda los retos de la era del Big Data, permitiendo una interpretación más sencilla, facilitando la comprensión visual de los datos, y contribuyendo a la calidad y eficiencia del análisis de datos. Su aplicabilidad transversal y su capacidad para revelar patrones en datos complejos subrayan su importancia y respaldan su continua relevancia en el análisis de datos moderno.

1.1. ¿En qué consiste el Análisis de Clustering?

El Análisis de Clustering, también conocido como análisis de agrupamiento, es una técnica estadística que se utiliza para clasificar elementos en grupos, de manera que los elementos dentro de cada grupo sean lo más similares posible y los grupos lo más diferente posible entre ellos. El clustering agrupa directamente las observaciones en función de sus similitudes.

El proceso de clustering comienza con la selección de una medida de similitud o distancia, como la distancia euclidiana o la de Minkowski. Luego, se utilizan algoritmos para agrupar los elementos en base a estas distancias, buscando la máxima homogeneidad interna y la mayor heterogeneidad entre los grupos.

Hay dos enfoques principales: el clustering jerárquico y el no jerárquico. En el jerárquico, no se conoce el número de grupos, sino que estos se forman secuencialmente, ya sea fusionando grupos pequeños en grupos más grandes (enfoque aglomerativo) o dividiendo un grupo grande en grupos más pequeños (enfoque divisivo). El no jerárquico, prefijamos un número de grupos finales, y se asigna cada observación a uno de estos grupos predefinidos basándose en la cercanía (o lejanía) entre observaciones.

Cada grupo formado en el análisis de clustering es una colección de datos que comparten características comunes. La interpretación de estos grupos puede revelar patrones, tendencias y relaciones subyacentes en los datos, los cuales podrían no ser evidentes sin el agrupamiento.

El éxito del clustering depende de la selección adecuada de la medida de distancia y del algoritmo de agrupamiento. Los resultados pueden variar significativamente con diferentes elecciones, haciendo que esta técnica sea tanto flexible como susceptible a la interpretación.

1.2. Ejemplos introductorios

Para ilustrar la utilidad de crear grupos en el análisis de datos, examinemos un conjunto de datos sobre pingüinos representado en un diagrama de dispersión, como se muestra en la Figura 1. Las variables analizadas son la longitud y la anchura del pico de los pingüinos. A primera vista, se esperaría una correlación

evidente entre estas medidas, pero el diagrama sugiere lo contrario, indicando una aparente falta de correlación. ¿Cómo interpretamos este fenómeno?

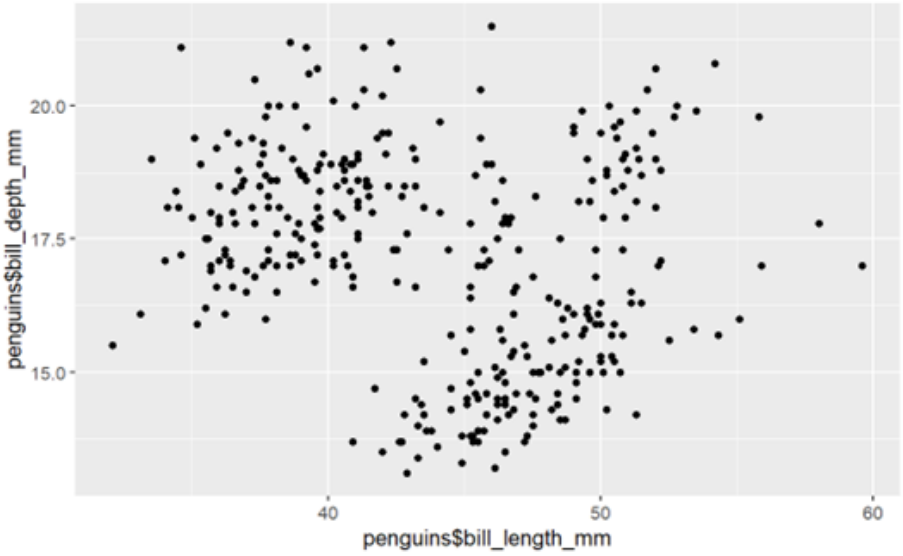


Figura 1: Distribución de los pingüinos

La clave para entender estos datos no reside en la ausencia de patrones, sino en la presencia de distintos grupos o 'tipos' dentro del conjunto de datos. Estos agrupamientos podrían representar diferentes subespecies de pingüinos, cada una con características únicas. Aunque las diferencias entre especies no se manifiestan directamente en las dos variables estudiadas, la observación de múltiples patrones o 'nubes' de datos, como se muestra en la Figura 2, aporta una mayor claridad al análisis.

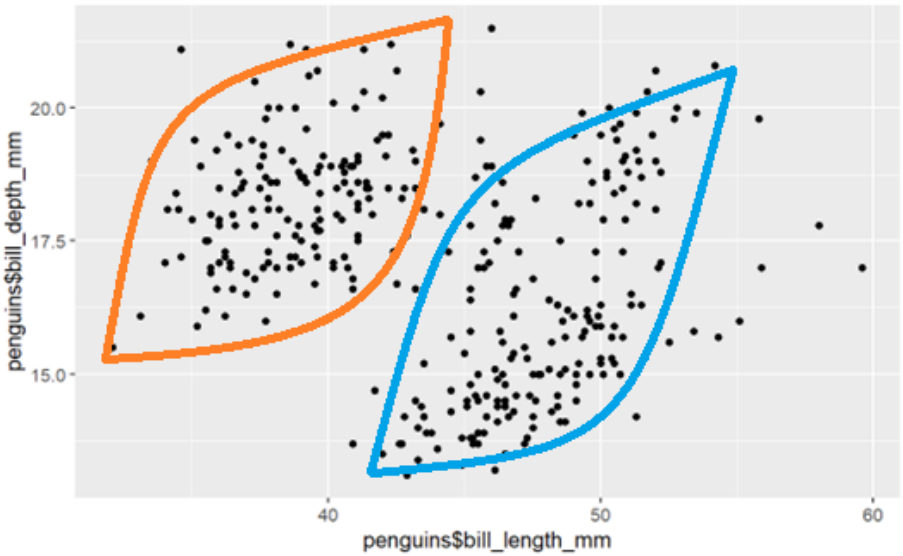


Figura 2: 2 posibles agrupaciones 'a ojo' de los pingüinos

Al identificar visualmente dos posibles agrupaciones en estos datos, hemos realizado un ejercicio preliminar de clustering. Aunque este enfoque intuitivo sugiere dos grupos, en realidad existen tres especies distintas de pingüinos en el conjunto de datos. Este caso sirve como una introducción a las técnicas de *clustering*, que aplicaremos de manera más sofisticada y precisa. En el análisis avanzado de clustering, utilizaremos más variables (*features*) y técnicas más refinadas, permitiéndonos descubrir agrupaciones en un espacio multidimensional con una mayor profundidad y precisión.

Ahora imaginemos otras tantas situaciones que se pueden dar en contextos cotidianos:

- El departamento de marketing de una empresa va a lanzar una campaña publicitaria sobre un nuevo producto. Para ello, desea tener a sus potenciales clientes agrupados según sus necesidades en los distintos aspectos de dicho producto.
- Se desea agrupar a los clientes de un banco para determinar diferentes perfiles a los que se les puede conceder un préstamo.
- En estudios genéticos en los que es habitual encontrar grupos de individuos con carga genética similar.

En todos los casos anteriores, nos encontramos con situaciones similares, en el sentido de que **nuestro objetivo es formar grupos de individuos con**

características similares con respecto a las puntuaciones que presentan en determinadas variables.

El Análisis de clustering (o simplemente *clustering*) es la técnica de Análisis Multivariante cuyo propósito es, a partir de un conjunto de individuos, crear "buenos" agrupamientos (clusters).

Por "buenos" agrupamientos entenderemos aquellos que cumplan al menos lo siguiente:

- Los individuos de cada grupo deben ser lo más parecidos que sea posible (homogeneidad interna).
- Los grupos deben ser lo más diferentes entre sí posible (heterogeneidad entre grupos).
- Generalmente los grupos obtenidos son mutuamente excluyentes (cada observación pertenece a un solo grupo).

En las técnicas de clustering *lo que nos va a interesar es crear agrupaciones a partir de los datos y no clasificarlos en unas categorías claramente preestablecidas,*

Una técnica básica para identificar *clusters* es la inspección gráfica, que implica observar visualmente las variables y agrupar observaciones cercanas, tal como se ejemplifica en nuestro estudio previo de los pingüinos. Esta técnica puede extenderse al análisis en planos factoriales después de un procesamiento adecuado de los datos. Sin embargo, en la práctica, nos enfrentamos a desafíos significativos debido al elevado número de observaciones y, más aún, a la complejidad añadida por la alta dimensionalidad de los datos (especialmente cuando $k \gg 2$).

Otro aspecto crucial en el *clustering* es la elección del método para medir la proximidad entre las observaciones. Existen diversas métricas y cada una puede influir notablemente en la formación de los grupos. Además, debemos decidir sobre el procedimiento específico de agrupación a utilizar, lo que puede dar lugar a una amplia variedad de resultados posibles.

En lo que respecta a las técnicas multivariantes, la representatividad de la muestra es de suma importancia. Asimismo, la presencia o ausencia de multicolinealidad (correlación entre variables) juega un papel fundamental en el análisis. A diferencia de otras técnicas de clasificación, en el *clustering* no conocemos de antemano ni el número de grupos en los que se divide la población ni los valores de las variables que definen cada grupo. Esta incertidumbre es lo que diferencia

al *clustering* de otros métodos de análisis y clasificación de datos.

La diferencia fundamental con respecto a otras técnicas de clasificación es que no se conoce de antemano el número de grupos en los que se divide a la población, ni el valor de la variable que identifica cada grupo.

---

2. ¿Qué requisitos precisan los datos originales para el Análisis de Clustering?

Para realizar un análisis de clustering efectivo, es importante que el conjunto de datos cumpla con ciertos requisitos. A continuación, se detallan algunos de estos:

- **Escala de las Variables**: En el clustering es crucial que las variables estén en la misma escala o sean estandarizadas, especialmente cuando las variables son de diferentes tipos o unidades de medida. Esto asegura que cada variable tenga un peso adecuado en el análisis.
- **Ausencia de Valores Atípicos**: Los valores extremos pueden distorsionar los grupos formados en el clustering. Es importante identificar y manejar adecuadamente estos valores atípicos antes de realizar el análisis.
- **Tratamiento de Datos Perdidos**: La presencia de datos faltantes puede afectar la calidad del clustering. Puede ser necesario aplicar técnicas de imputación de datos.
- **Minimizar la Correlación entre Variables**: Es recomendable que las variables no estén fuertemente correlacionadas. Las variables correlacionadas pueden aportar información redundante, lo que podría conducir a una sobrerrepresentación de ciertas características y afectar la formación de los grupos. Considerar la reducción de dimensionalidad o la selección de variables puede ser útil en tales casos.

3. ¿Cómo se hace un Análisis de *clustering?*

El análisis de clustering implica una serie de pasos estructurados para formar grupos homogéneos de datos. Estos pasos se pueden resumir de la siguiente manera:

1. **Evaluación del Parecido entre Observaciones:**
   - Determinar la proximidad o similitud entre las observaciones utilizando medidas de distancia.
2. **Evaluación del Parecido entre Variables:**
   - Evaluar la similitud o correlación entre las variables, si el enfoque del clustering es sobre las características en lugar de las observaciones.
3. **Selección del Tipo de Agrupación:**
   - Decidir entre enfoques de agrupación jerárquicos (donde el número de clusters se determina durante el análisis) y no jerárquicos (donde se debe definir el número de clusters de antemano).

4. **Elección del Algoritmo de Agrupación:**
   - Seleccionar un algoritmo específico de agrupación, como k-means, agrupamiento jerárquico, DBSCAN, etc., en función del tipo de agrupación elegido.
5. **Determinación del Número Óptimo de Clusters:**
   - Establecer criterios para definir el número adecuado de clusters, especialmente en métodos no jerárquicos.
6. **Interpretación y Validación de los Clusters Formados:**
   - Analizar e interpretar los grupos resultantes para validar su significado y relevancia.

Este esquema proporciona una guía general para realizar un análisis de clustering, aunque los detalles específicos pueden variar según el conjunto de datos y los objetivos del estudio.

3.1. Medidas de Distancia entre Observaciones

Para evaluar la semejanza entre dos observaciones en el análisis de clustering, empleamos medidas de distancia y similitud. La idea central es que dos observaciones son más parecidas entre sí cuanto menor es la distancia o mayor es la similitud que presentan.

Consideremos dos observaciones individuales denotadas como (x, y), que constan en p variables, es decir, $(x_i, y_i) = \{(x_1, x_2, \ldots, x_p), (y_1, y_2, \ldots, y_p)\}$. Entre las medidas de distancia más utilizadas para variables numéricas, destacan las siguientes:

- **Distancia Euclídea**: Es la medida de distancia más común y se calcula como:

$$d_{\text{Euclídea}}(x, y) = \sqrt{\sum_{i=1}^{p} (x_i - y_i)^2} \tag{1}$$

- **Distancia Minkowski**: Una forma generalizada que abarca distintos tipos de distancias, incluyendo la Euclídea y la Manhattan. Se define como:

$$d_{\text{Minkowski}}(x, y) = \left(\sum_{i=1}^{p} |x_i - y_i|^p \right)^{1/p} \tag{2}$$

  En particular, para $p = 2$ obtenemos la distancia Euclídea, y para $p = 1$, la distancia Manhattan.

Nota: Aunque las distancias Euclídea y Manhattan son las más utilizadas en el análisis de clustering, ambas son casos particulares de la distancia Minkowski.

3.2. Medidas de Distancia entre Variables

En el análisis de clustering, además de agrupar observaciones, puede ser útil agrupar variables que exhiban comportamientos similares o estén correlacionadas entre sí. Para ello, se emplean medidas de distancia basadas en coeficientes de correlación. Estas medidas ayudan a identificar la similitud en el comportamiento de diferentes variables a lo largo del conjunto de datos. A continuación, describimos algunas de estas medidas:

- **Distancia de Correlación de Pearson**: Esta medida evalúa la similitud entre dos variables basándose en la correlación lineal de Pearson. Se calcula como:

$$d_{\text{Pearson}}(x, y) = 1 - \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2 \sum_{i=1}^{n}(y_i - \bar{y})^2}} \tag{3}$$

  donde x y y son observaciones individuales, y $\bar{x}$ y $\bar{y}$ son las medias de las muestras x e y, respectivamente.

  El resultado de la correlación de Pearson es un valor que varía entre -1 y 1, donde:

  - Un valor de -1 indica una relación lineal perfecta negativa.
  - Un valor de 1 indica una relación lineal perfecta positiva.
  - Un valor de 0 sugiere que no hay relación lineal.

- **Distancia de Correlación de Spearman**: Utilizada cuando los datos son ordinales o no cumplen con los supuestos de la correlación de Pearson. La correlación de Spearman se basa en los rangos de los datos, lo que la hace menos sensible a valores atípicos y apropiada para relaciones monótonas no lineales.

$$\rho_{\text{Spearman}} = 1 - \frac{6 \sum_{i=1}^{n} d_i^2}{n(n^2 - 1)} \tag{4}$$

  Donde:

  - d_i representa la diferencia entre los rangos de cada par de observaciones emparejadas.
  - n es el número total de pares de observaciones.

  El resultado de la correlación de Spearman es un valor que varía entre -1 y 1, donde:

  - Un valor de -1 indica una relación monótona perfecta negativa.
  - Un valor de 1 indica una relación monótona perfecta positiva.

- Un valor de 0 sugiere que no hay relación monótona.

Una relación monótona significa que las variables tienden a cambiar juntas, pero no necesariamente a un ritmo constante.

- **Distancia de Correlación de Kendall**: Conocida como el coeficiente de concordancia de Kendall, esta medida evalúa la concordancia entre dos conjuntos de datos basándose en la concordancia o discordancia de los pares de observaciones.

$$\tau_{\text{Kendall}} = \frac{\text{Nº de pares concordantes} - \text{Nº de pares discordantes}}{\text{Nº total de pares}} \tag{5}$$

Un par de observaciones se considera concordante si el orden relativo de las observaciones es el mismo en ambos conjuntos de datos. Por otro lado, un par es discordante si el orden relativo es diferente en los dos conjuntos de datos. Si tenemos n observaciones, existen $\frac{n(n-1)}{2}$ pares posibles.

El resultado del coeficiente de asociación de Kendall es un valor que varía entre -1 y 1, donde:

- Un valor de -1 indica una falta de concordancia completa entre los conjuntos de datos.
- Un valor de 1 indica una concordancia perfecta.
- Un valor de 0 sugiere que no hay una concordancia ni discordancia sistemática.

Estas medidas de distancia entre variables son fundamentales para agrupar variables en el análisis de clustering, especialmente cuando se busca entender las relaciones subyacentes entre diferentes características en un conjunto de datos.

4. Algoritmos de Clasificación Jerárquicos

Los métodos de clasificación jerárquica organizan los datos en una estructura arborescente, comenzando con grupos que son observaciones individuales y agrupándolos progresivamente en clusters de niveles superiores. A diferencia de los métodos no jerárquicos, en la clasificación jerárquica no se conoce de antemano el número final de clusters.

Los pasos para realizar un *Clustering* Jerárquico incluyen:

1. Iniciar con tantos grupos como observaciones existan.
2. Crear una matriz simétrica de dimensión $n \times n$ que represente las distancias entre todos los pares de observaciones. Esta distancia debe haber sido previamente definida.
3. Agrupar las dos observaciones más cercanas, reduciendo el número total de clusters en uno.
4. Calcular una nueva matriz de distancias entre los clusters formados, eligiendo un método adecuado para calcular la distancia entre clusters.
5. Repetir el proceso de agrupación.
6. Continuar este proceso hasta que todas las observaciones estén agrupadas en un único cluster.

Es crucial definir claramente tanto la distancia entre observaciones individuales como la distancia entre clusters. Por lo tanto, con el mismo conjunto de datos y variando estas definiciones, se pueden obtener diferentes clasificaciones jerárquicas.

4.1. Evaluando la distancia entre cluster o grupos de observaciones

Supongamos que tenemos una agrupación denominada C_k y otra agrupación C_l, los algoritmos más comunes son:

- **Enlace simple o del vecino más cercano**: Se establece la siguiente distancia:

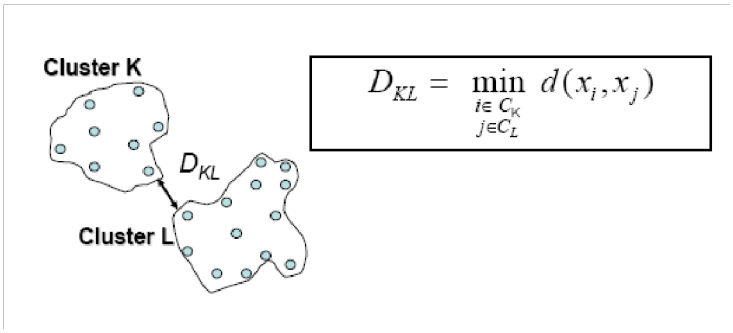


Figura 3: Distancia entre cluster: Vecino más cercano

Es decir, la distancia entre dos grupos es la distancia entre las dos observaciones más cercanas, pertenecientes cada una de ellas a un grupo distinto. Tienden a crear grupos con muchas observaciones.

- **Enlace completo o del vecino más alejado**: La distancia elegida entre dos cluster será:

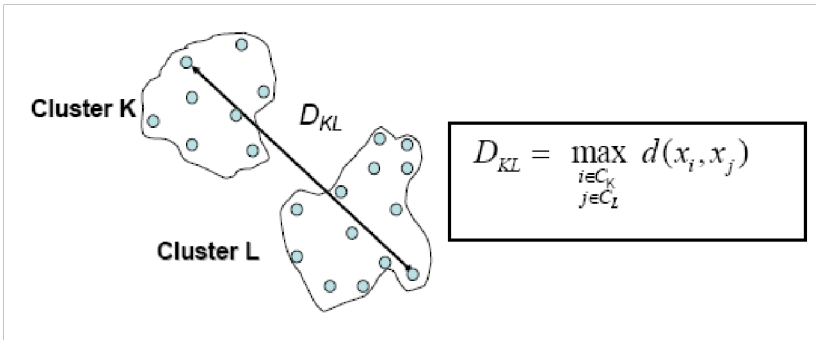


Figura 4: Distancia entre cluster: Vecino más lejano

Luego, la distancia entre dos grupos, es la distancia entre las dos observaciones más alejadas de cada uno de los dos grupos. Con este método, los grupos formados son más compactos que los obtenidos con el método del vecino más próximo.

- **Enlace medio**: La distancia entre dos agrupaciones es la distancia media entre una observación de la agrupación C_k y otra observación de la agrupación C_l.

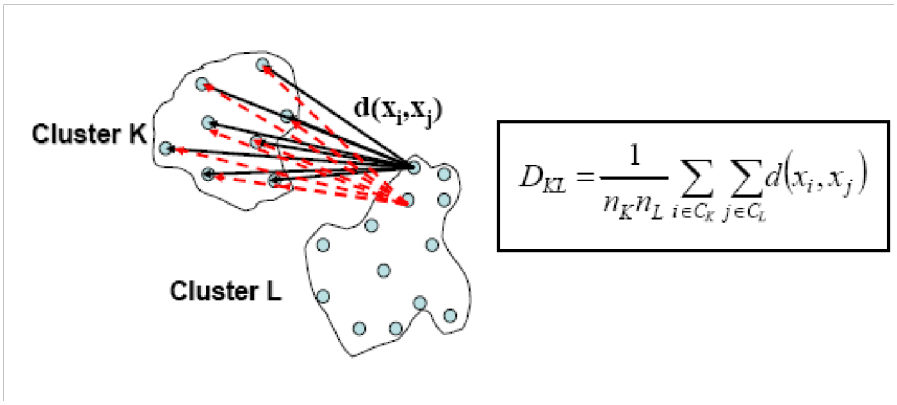


Figura 5: Distancia entre cluster: enlace medio

El método de enlace medio tiende a crear grupos que son más equilibrados en términos de tamaño y forma, es decir, grupos con varianza similar.

- **Distancia entre centroides**: La distancia entre grupos será la distancia entre los centroides, que representarán el vector medio obtenido en las p variables para todos los individuos que formen parte del grupo.

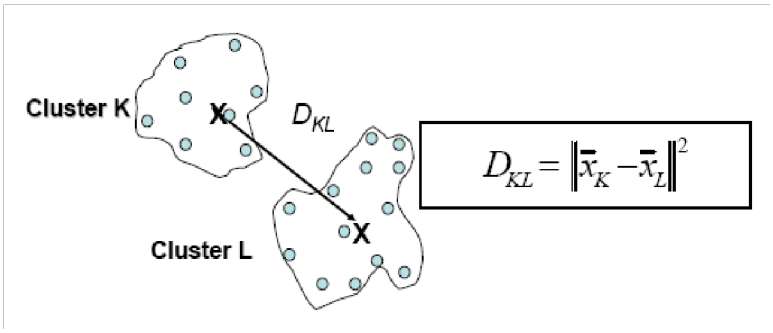


Figura 6: Distancia entre centroides

4.2. Distancia de Ward o de la Mínima Varianza

El método de la Distancia de Ward selecciona, en cada nivel del proceso de agrupación, la combinación de clusters que minimiza el incremento de la variabilidad total dentro de los clusters resultantes. Este enfoque se basa en la suma de cuadrados de las distancias euclidianas entre las observaciones y el centroide del grupo fusionado.

La fórmula de la Distancia de Ward es la siguiente:

$$d_{\text{Ward}}(C_i, C_j) = \frac{n_i n_j}{n_i + n_j} \sum_{x \in C_i \cup C_j} \|x - \mu_{C_i \cup C_j}\|^2$$

Donde:

- n_i y n_j representan el número de observaciones en los grupos C_i y C_j, respectivamente.
- x son las observaciones en la unión de los clusters C_i y C_j.
- $\mu_{C_i \cup C_j}$ es la media de las observaciones en la unión de los conglomerados C_i y C_j.
- La suma se realiza sobre todas las observaciones en la unión de C_i y C_j, calculando la distancia euclidiana al cuadrado entre cada observación y el centroide del grupo fusionado.

Este método es especialmente eficaz para identificar clusters compactos y homogéneos, minimizando la varianza interna de los grupos resultantes.

La distancia de Ward se usa comúnmente en agrupaciones jerárquicas aglomerativas para determinar qué grupos deben fusionarse para crear una jerarquía. Algunas de sus ventajas son:

- Minimización de la varianza: el método Ward tiene como objetivo minimizar la varianza dentro de cada grupo. Evalúa el aumento de la varianza que resulta de la fusión de conglomerados y busca minimizar este aumento. Esto puede dar lugar a agrupaciones más compactas y homogéneas.
- Equilibrio de tamaños de conglomerados: el método de Ward tiende a producir conglomerados de tamaños aproximadamente iguales. Esto puede resultar ventajoso en determinadas aplicaciones en las que es deseable tener grupos de tamaños similares.
- Sensible a la forma de los cúmulos: el método Ward es relativamente sensible a la forma de los cúmulos/grupos y, a menudo, funciona bien cuando los cúmulos son globulares o elípticos. Esto lo hace adecuado para conjuntos de datos donde los grupos subyacentes tienen estas estructuras determinadas.
- Estructura jerárquica: el método Ward produce naturalmente una estructura de agrupamiento jerárquica, representada en un dendrograma. Esto puede resultar útil para visualizar relaciones entre clústers en diferentes niveles de granularidad.
- Menos Sensible a las Condiciones Iniciales: A diferencia de otros métodos de clustering, donde la elección inicial de algún parámetro puede afectar significativamente los resultados, el método de Ward es menos susceptible a estas condiciones iniciales. Esto se debe a que su enfoque se basa en

minimizar un criterio objetivo (la suma de cuadrados dentro de los clusters) en cada paso, lo que lo hace más estable y consistente en diferentes ejecuciones, independientemente de las inicializaciones.

Tras ver las distancias anteriores, uno podría preguntarse: **¿Cúal es la distancia entre clusters más adecuada?**

No existe una respuesta exacta a esta pregunta (aunque hayamos resaltado las ventajas de la de Ward). Una posibilidad es probar varias distancias y tomar una decisión en función de los resultados que se obtengan. Si varios métodos nos dan agrupaciones similares, se puede pensar que existe una forma natural de formarse grupos de observaciones. En la práctica el método más utilizado es el de Ward, porque nos asegura la máxima homogeneidad dentro de los cluster.

Llevamos a cabo un ejemplo de agrupación mediante un algoritmo jerárquico, sobre un conjunto de seis observaciones de las que presentamos sus distancias. Utilizamos como distancia entre cluster la distancia simple o del vecino más cercano. El proceso a seguir, es el indicado en la Figura 7.

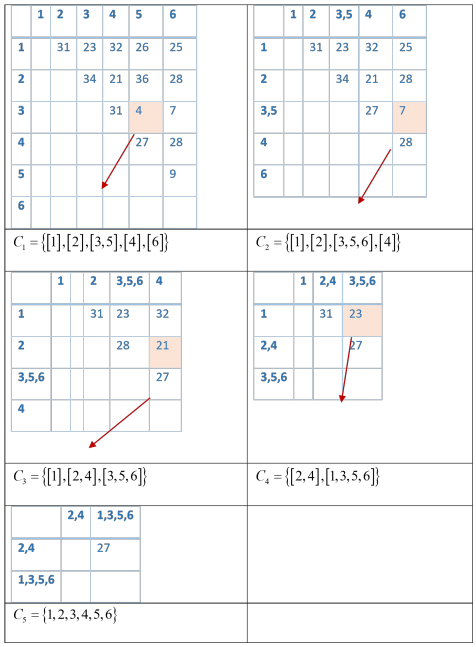


Figura 7: Distancia entre cluster

4.3. Resultados del Clúster Jerárquico: El Dendrograma

El análisis clúster jerárquico se ilustra con frecuencia mediante un dendrograma, como se muestra en la Figura 16. Este gráfico tiene la estructura de un árbol y visualiza el proceso de formación de grupos y la secuencia de fusiones entre ellos. Las alturas de las conexiones en el dendrograma representan las distancias entre los grupos que se combinan, proporcionando una medida cuantitativa de la disimilitud entre conjuntos de observaciones.

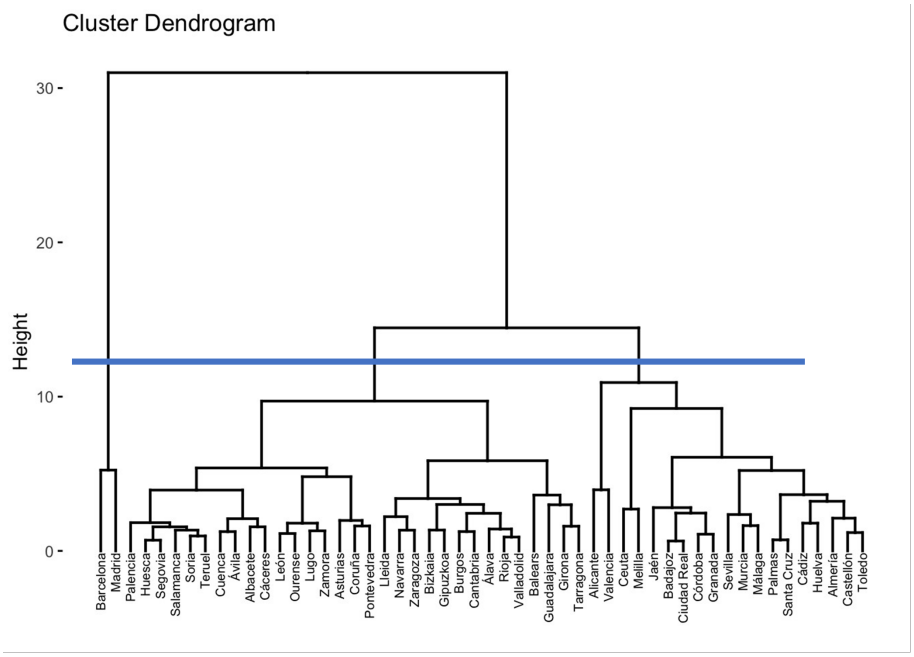


Figura 8: Dendograma.

Cada observación comienza como su propio grupo y, paso a paso, se van uniendo para formar clústers más grandes. La altura en la que dos grupos se unen, indicada por los valores de la escala vertical, refleja la distancia entre ellos; así, los grupos que se unen a alturas menores son más similares. El dendrograma facilita la selección del número de clústers cortando el árbol a una altura deseada. Esto nos permite ajustar el nivel de detalle con el que agrupamos los datos, lo que se conoce como granularidad de la agrupación.

La granularidad en un dendrograma se refiere al nivel de detalle o especificidad de los grupos formados. Un alto nivel de granularidad significa que existen muchos grupos pequeños y bien definidos, mientras que un bajo nivel de granularidad resulta en menos grupos más grandes y generales. Al cortar el

dendrograma a diferentes alturas, se determina la granularidad de los clústers:

- Una altura baja en el dendrograma produce una gran cantidad de clústers pequeños y específicos, reflejando una alta granularidad.
- Una altura mayor resulta en un menor número de clústers más grandes y menos definidos, indicando una baja granularidad.

En nuestro ejemplo, el corte sugerido propine $k = 3$ clústers, donde uno de ellos cuenta solo con dos observaciones (baja granularidad) y los otros dos sugieren agrupaciones con mayor granularidad.

5. Algoritmos de Clasificación No Jerárquicos

Los algoritmos de agrupación no jerárquicos requieren definir previamente el número de grupos (K) en los que se dividirán las observaciones. Por ello, suele preceder un análisis jerárquico para estimar el número más adecuado de clusters. Estos son los pasos típicos que se siguen en un análisis no jerárquico:

1. Elegir K observaciones como centroides iniciales, correspondientes a la cantidad de clusters deseados.
2. Asignar cada observación restante al cluster cuyo centroide sea el más cercano.
3. Reasignar las observaciones a los K clusters según una regla de parada predefinida.
4. Detener el proceso cuando no haya reasignaciones o se cumpla la regla de parada; de lo contrario, regresar al paso 2.

Los métodos para seleccionar los centroides iniciales varían e incluyen:

- Tomar las K primeras observaciones sin datos faltantes.
- Escoger observaciones que maximicen una distancia predefinida entre centroides sucesivos.
- Seleccionar K observaciones al azar.
- Optar por centroides que estén lo más lejanos posible entre sí.
- Emplear centroides definidos por el investigador.

Una vez identificados los centroides, se forman los clusters iniciales asignando cada una de las $N - K$ observaciones restantes al cluster con el centroide más próximo.

La reasignación de observaciones se puede regir por varias reglas, como:

1. Calcular el centroide de cada cluster y reasignar observaciones al cluster con el centroide más cercano. **Los centroides se recalculan tras cada reasignación y el proceso se repite hasta que el cambio en los centroides es menor que un umbral preestablecido por un criterio de convergencia**.

2. Reasignar observaciones minimizando un criterio estadístico, como la suma de cuadrados dentro de los clusters.

Los criterios de parada son esenciales en los algoritmos de clusterización no jerárquicos y suelen basarse en la estabilidad de los centroides o en la minimización de una función de coste como la suma de cuadrados dentro de los clusters. Estos criterios garantizan que el algoritmo se detenga cuando se alcanza una solución óptima o cuando se satisfacen las condiciones de convergencia.

5.1. Criterios de Parada en Clustering No Jerárquico

El criterio de parada en un algoritmo no jerárquico es un aspecto crucial que determina cuándo debe finalizar el proceso de reasignación de observaciones. Un criterio de parada común es la estabilidad de los centroides de los clusters, que ocurre cuando los centroides no cambian significativamente entre iteraciones sucesivas, indicando que se ha alcanzado una solución estable. Otro criterio se relaciona con la función objetivo, como la suma de cuadrados dentro de los clusters, que debe minimizarse. Cuando los cambios en esta suma de cuadrados son menores que un umbral definido, el algoritmo se detiene, sugiriendo que se ha encontrado una agrupación óptima.

1. **Convergencia de Centroides**: El proceso se detiene cuando la posición de los centroides no varía o los cambios son menores que un límite de tolerancia después de una iteración.

2. **Minimización de la Función Objetivo**: El algoritmo se detiene cuando la mejora de la función objetivo entre iteraciones sucesivas es menor que un umbral especificado, indicando que se ha alcanzado una optimización suficiente.

3. **Número Máximo de Iteraciones**: También se puede establecer un número máximo de iteraciones después del cual el algoritmo se detendrá, independientemente de si se han alcanzado otros criterios de parada.

Estos criterios de parada aseguran que el algoritmo no continúe indefinidamente y que los clusters resultantes reflejen una estructura significativa dentro de los datos.

5.2. Diferencias entre Análisis Jerárquico Prefijando el Número de Clusters y Análisis No Jerárquico

En el campo del análisis de clusters, existen dos enfoques principales: el análisis jerárquico, que construye una jerarquía de posibles agrupaciones, y el

análisis no jerárquico, que asigna puntos a clusters de manera directa. A continuación, se detallan las diferencias clave entre estos dos métodos, especialmente cuando se prefija el número de clusters en el análisis jerárquico.

Análisis Jerárquico Prefijando Grupos:

- *Estructura Jerárquica:* Primero se construye una jerarquía completa de agrupaciones, representada a menudo como un dendrograma.
- *Decisión de Número de Clusters:* Después de construir la jerarquía completa de agrupaciones, se elige el número de clusters deseado. Este proceso implica seleccionar un nivel en la jerarquía, usualmente a través de un criterio como la distancia máxima entre clusters, para determinar el corte que resultará en el número especificado de grupos. Este enfoque permite una gran flexibilidad en la elección del número de clusters basado en la estructura jerárquica de los datos.
- *Relaciones entre Clusters:* La jerarquía revela no solo los grupos, sino también cómo están relacionados entre sí.
- *Flexibilidad en la Formación de Clusters:* Los clusters se forman basándose en la similitud y proximidad dentro de la jerarquía, no en un centro específico.

Análisis No Jerárquico (como K-means):

- *Asignación Directa a Clusters:* Cada punto de datos se asigna a un cluster basándose en su cercanía a un centroide, sin construir una jerarquía.
- *Número de Clusters Predeterminado:* El número de clusters se decide de antemano y la formación de estos se hace directamente.
- *Sin Información sobre la Relación entre Clusters:* No proporciona información inherente sobre cómo los diferentes clusters se relacionan o se comparan entre sí.
- *Clusters Basados en Centroides:* Los clusters están definidos por sus centroides.

6. Determinación de número de clusters K

En el análisis de clustering, uno de los desafíos más significativos es determinar el número óptimo de clusters (K), dicho desafío esta instrínsicamente relacionado con la evaluación de la calidad de nuestras agrupaciones. Ya que la bondad de ajuste de los clusters nos proporciona información valiosa sobre la adecuación del número de clusters elegido.

Para evaluar una solución de agrupación, se pueden calcular varias medidas relacionadas con la variabilidad dentro de los grupos, entre grupos y la variabilidad total.

1. **Variabilidad dentro de los conglomerados (suma de cuadrados dentro de los grupo, WCSS):**

   El WCSS mide la variabilidad dentro de cada grupo. Cuantifica qué tan lejos están los puntos de datos dentro de un grupo del centroide del grupo.

   $$\text{WCSS} = \sum_{i=1}^{K} \sum_{j=1}^{n} d(x_{ij}, c_i)^2, \tag{6}$$

   donde

   - K es el número de conglomerados.
   - n es el número de puntos de datos en el grupo i.
   - $d(x_{ij}, c_i)$ es la distancia entre el punto de datos x_{ij} y el centroide del grupo i.

2. **Variabilidad entre conglomerados (suma de cuadrados entre conglomerados, BCSS):** El BCSS mide la variabilidad entre los centroides de los conglomerados. Cuantifica qué tan lejos están los centroides del grupo entre sí.

   $$\text{BCSS} = \sum_{i=1}^{K} n_i \cdot d(c_i, c)^2, \tag{7}$$

   donde

   - n_i es el número de datos en el grupo i.
   - $d(c_i, c)$ es la distancia entre el centroide del grupo i y el centroide general c.

3. **Variabilidad total (Suma total de cuadrados, TSS):**

   El TSS mide la variabilidad total de los datos y es la suma de las distancias al cuadrado entre los puntos de datos y el centroide general.

   $$\text{TSS} = \sum_{i=1}^{K} \sum_{j=1}^{n} d(x_{ij}, c)^2, \tag{8}$$

   donde

   - n es el número total de datos.

- $d(x_{ij}, c)$ es la distancia entre el punto de datos x_{ij} y el centroide general c.

4. **Coeficiente de determinación (R-cuadrado o R^2):**

   R^2 es una medida de qué tan bien el agrupamiento explica la variabilidad de los datos. Se calcula como la relación entre BCSS y TSS.

$$R^2 = \frac{\text{BCSS}}{\text{TSS}} = 1 - \frac{\text{WCSS}}{\text{TSS}}. \tag{9}$$

Estas medidas pueden ayudarle a evaluar la calidad de su solución de clustering. Los valores de R^2 más altos indican una mejor solución de agrupación, ya que explican una mayor parte de la variabilidad total de los datos. Los valores más bajos de WCSS sugieren que los puntos de datos están más cerca de sus respectivos centroides de grupo.

Además del análisis Jerárquico, existen procedimientos (además de las gráficas) que nos ayudan a decidir el número más adecuado de cluster. Técnicas como el método de codo:

El **método del codo** es una técnica popular para determinar el número óptimo de conglomerados (K) en un algoritmo de agrupamiento. Ayuda a encontrar el punto en el que agregar más grupos no mejora significativamente la calidad de la agrupación y, a menudo, se lo denomina "punto codo.[en] un gráfico. Así es como funciona:

1. Calcula la WCSS (suma de cuadrados dentro de un grupo) para diferentes valores de K. La WCSS mide la variabilidad dentro de cada grupo y cuantifica qué tan lejos están los puntos de datos dentro de un grupo del centroide del grupo.

2. Trazar los valores de WCSS frente al número de grupos (K). Normalmente verá una curva que comienza en alto y disminuye a medida que K aumenta. El punto donde la curva comienza a doblarse y formar una forma de 'codo' es el número óptimo de grupos.

El 'punto de codo' es donde el WCSS comienza a disminuir a un ritmo más lento, lo que indica que agregar más grupos no reduce significativamente el WCSS. Esta suele ser una buena opción para determinar la cantidad de cluster que se utilizarán en la solución (agrupación final).

El método del codo es un método heurístico útil para elegir el número de conglomerados, pero es importante recordar que no siempre es definitivo, y que también se deben considerar otros factores, como el conocimiento del dominio y los requisitos específicos del problema, al seleccionar la k óptima.

El **método de la silueta:** El coeficiente de silueta es una medida que se utiliza para evaluar y graficar la calidad de los clusters. Para cada observación, el coeficiente de silueta es un valor que indica cuán similar es una observación a las otras observaciones en su propio cluster en comparación con las observaciones en los clusters más cercanos. El cálculo del coeficiente de silueta para cada observación se realiza de la siguiente manera:

1. **Calcular la distancia media** $a(i)$: Para cada observación i, calcula la distancia media de i a todas las otras observaciones en el mismo cluster. Esto mide cuán bien se agrupa la observación en su propio cluster.

2. **Calcular la distancia media** $b(i)$ **al cluster más cercano**: Para cada observación i, calcula la distancia media de i a todas las observaciones en el cluster más cercano, es decir, el cluster al que i no pertenece y tiene la menor distancia media. Esto mide cuán bien se separa la observación de los clusters más cercanos.

3. **Calcular el coeficiente de silueta** $s(i)$: Para cada observación i, el coeficiente de silueta se calcula usando la fórmula:

$$s(i) = \frac{b(i) - a(i)}{\text{máx}\{a(i), b(i)\}}$$

   Donde:

   - $a(i)$ es la distancia media dentro del cluster de la observación i.
   - $b(i)$ es la menor distancia media de i a cualquier otro cluster del que i no es miembro.

   El valor de $s(i)$ varía entre -1 y 1:

- Un valor cercano a $+1$ indica que la observación está bien agrupada y lejos de los clusters vecinos.

- Un valor cercano a 0 indica que la observación está cerca de la frontera de decisión entre dos clusters vecinos.

- Un valor cercano a -1 indica que la observación puede haber sido asignada al cluster incorrecto.

El gráfico resultante del método de la silueta nos proporcionará una visualización clara del puntaje de silueta promedio para cada valor de K, lo que nos ayudará a seleccionar el número de clusters que ofrece el mejor equilibrio.

El cálculo del coeficiente de silueta para cada observación puede ser computacionalmente costoso, especialmente para grandes conjuntos de datos, pero proporciona una evaluación detallada de la calidad del clustering a nivel individual.

7. Caracterización de los Clústeres

Una vez establecida la partición en clústers, el paso siguiente es caracterizar y comprender cada clúster. Esta caracterización se basa en un análisis descriptivo de las variables utilizadas durante el análisis de clúster. Estos estadísticos descriptivos facilitan una primera aproximación para interpretar las propiedades distintivas de cada clúster en relación con las variables originales.

Es esencial identificar los miembros de cada clúster. Sin embargo, para obtener una comprensión más profunda de las diferencias entre clústers, resulta más informativo examinar los estadísticos de resumen para las variables originales. Al hacerlo, es posible interpretar los clústers en términos de la magnitud y la relevancia de las variables dentro de cada clúster. A continuación, se presentan algunos pasos y métodos comunes para la caracterización de clústers:

1. **Identificación de Miembros:** Determinar los elementos que componen cada clúster, lo que puede involucrar la creación de listados o tablas de pertenencia.

2. **Estadísticos Descriptivos:** Calcular medidas de tendencia central y dispersión, como la media, mediana, varianza y desviación estándar, para cada variable dentro de los clústers.

3. **Comparación entre Clústeres:** Utilizar los estadísticos descriptivos para comparar cómo se diferencian los clústers en términos de las variables analizadas.

4. **Perfilado de Clústeres:** Construir perfiles o descripciones cualitativas de cada clúster basados en los estadísticos obtenidos, lo que puede incluir gráficas o tablas resumen.

5. **Análisis de Variables Significativas:** Identificar las variables que más contribuyen a la formación de los clústers y que mejor los describen.

6. **Visualización de Datos:** Emplear técnicas de visualización como gráficos de barras, histogramas o boxplots para ilustrar las características de cada clúster.

Esta caracterización exhaustiva no solo sirve para entender mejor cada clúster, sino también para informar decisiones relacionadas con la estrategia y acciones a seguir basadas en la agrupación realizada.

8. Ejemplo de Análisis Cluster con Python

Para ilustrar la aplicación práctica del análisis de clustering, consideraremos un conjunto de datos que contiene la esperanza de vida de diferentes países y regiones. Este ejemplo práctico de análisis de clúster tiene como fin explorar

y clasificar países en función de su esperanza de vida en diferentes etapas, diferenciada por sexo. A través de este análisis, buscamos identificar patrones y agrupaciones que nos permitan entender mejor las diferencias y similitudes en la esperanza de vida a nivel mundial.

Utilizaremos Python, un poderoso lenguaje de programación y sus bibliotecas de análisis de datos para llevar a cabo este estudio. El proceso se desglosará en las siguientes etapas clave:

1. Preparación de los datos: Cargar y visualizar la información contenida en el archivo `EsperanzaVida.xlsx`, estableciendo una comprensión inicial del conjunto de datos.

2. Análisis exploratorio: Implementar un mapa de calor para identificar relaciones preliminares y patrones dentro del conjunto de datos.

3. Cálculo de distancias: Generar y visualizar una matriz de distancias para evaluar la similitud entre las observaciones.

4. Estandarización de datos: Preparar los datos para el análisis de clúster mediante la estandarización, facilitando así la comparabilidad entre las variables.

5. Análisis de clúster jerárquico: Aplicar un enfoque jerárquico para agrupar los países y visualizar los resultados con un dendrograma.

6. Selección del número de clústers: Utilizar métodos como el del codo y la silueta para determinar el número óptimo de clústers.

7. Desarrollar un análisis no jerárquico mediante el algoritmo $k - means$.

8. Caracterización de clústers: Describir las propiedades y características distintivas de cada clúster identificado.

Cada paso estará acompañado de código Python, ilustraciones y explicaciones que guiarán al lector a través de las fases del análisis. Al final del ejemplo, seremos capaces de caracterizar y diferenciar clústers de países según sus perfiles de esperanza de vida.

Lo primero por tanto, es importar las librerías que vamos a utilizar y la carga de los datos

```
# Importamos las bibliotecas necesarias
import os
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
from scipy.spatial import distance
from sklearn.preprocessing import StandardScaler
import scipy.cluster.hierarchy as sch
from scipy.cluster.hierarchy import fcluster
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from scipy.spatial.distance import squareform
from sklearn.metrics import silhouette_score
from sklearn.metrics import silhouette_samples

# Establecemos la ubicacion de nuestro directorio de trabajo
os.chdir('G:\Mi unidad\Apuntes titulo propio1\MasterDani\Material
    Definitivo\M2\Datos')

# Leemos el archivo de Excel en un DataFrame
df = pd.read_excel('EsperanzaVida.xlsx')

# Visualizamos las primeras filas del DataFrame para verificar los
    datos
df.head()

# Configuramos 'PAIS' como el indice del DataFrame
df = df.set_index('PAIS')
```

ndex	PAIS	m0	m25	m50	m75	w0	w25	w50	w75
0	Algeria	63	51	30	13	67	54	34	15
1	Cameroon	34	29	13	5	38	32	17	6
2	Madagascar	38	30	17	7	38	34	20	7
3	Mauritius	59	42	20	6	64	46	25	8
4	Reunion	56	38	18	7	62	46	25	10

Figura 9: Un vistazo a los datos sobre Esperanza de vida

Construimos un mapa de calor, donde se representan de manera ordenada los valores observados, así como un proceso de cluster jerárquico donde se muestran los diferentes pasos iterativos de unión de observaciones.

```
# Creamos un mapa de calor (heatmap)
sns.clustermap(df, cmap='coolwarm', annot=True)
# Agregamos un titulo al grafico
plt.title('Mapa de Calor')
# Etiquetamos el eje x
plt.xlabel('E(vida) por edad y sexo')
# Etiquetamos el eje y
```

```
plt.ylabel('Pais')
# Mostramos el grafico
plt.show()
```

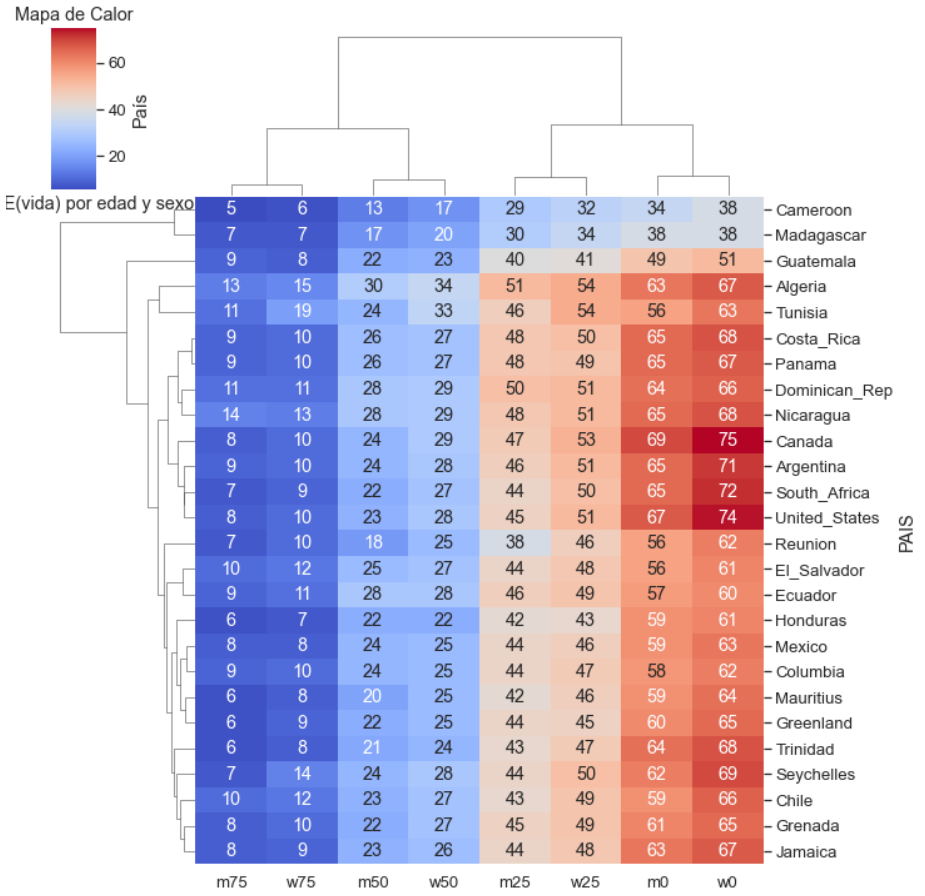


Figura 10: clustermap

La función *clustermap()* crea un mapa de calor, reorganizando las filas y columnas según una agrupación jerárquica. Puede ayudar a revelar patrones y relaciones dentro de los datos. Pero también nos permite encontrar parecidos entre las variables, mostrándonos además un dendrograma de agrupación posible, tanto entre individuos como entre variables.

Una vez representados los datos y un proceso jerárquico preliminar vamos a calcular la matriz de distancias (Euclídeas) entre las observaciones (con los valores sin estandarizar):

```
from scipy.spatial import distance

# Calculate the pairwise Euclidean distances
distance_matrix = distance.cdist(df, df, 'euclidean')

# The distance_matrix is a 2D array containing the Euclidean
    distances between all pairs of observations.

distance_small = distance_matrix[:5, :5]
#Index are added to the distance matrix
distance_small = pd.DataFrame(distance_small, index=df.index
    [:5], columns=df.index[:5])

distance_small_rounded = distance_small.round(2)
print("Distance Matrix fr:"distance_small_rounded)
```

Cuadro 1: Matriz de distancia entre países

Country	Algeria	Cameroon	Madagascar	Mauritius	Reunion
Algeria	0.00	58.08	52.65	21.19	24.35
Cameroon	58.08	0.00	7.14	42.24	38.03
Madagascar	52.65	7.14	0.00	37.96	33.81
Mauritius	21.19	42.24	37.96	0.00	6.16
Reunion	24.35	38.03	33.81	6.16	0.00

Por si se desea profundizar, la función anterior *cdist()* admite las siguientes distancias: *Euclidean, Manha-ttan, Minkowski, Chebyshev, Cosine, Hamming, Jaccard, Correlation, Mahalanobis, Canberra.*

Representemos ahora mediante escalas de color la distancia entre todas las observaciones, esto nos ayudará a detectar grupos de observaciones con grandes distancias entre sí.

```
# Representamos graficamente la matriz de distancias

# Crea una nueva figura para el grafico con una dimension
    especifico
plt.figure(figsize=(10, 8))
# Genera un mapa de calor usando Seaborn
# - 'distance_df': DataFrame de pandas que contiene los datos para
    el mapa de calor.
# - 'annot=False': No muestra las anotaciones (valores de los datos
    ) en las celdas del mapa.
# - 'cmap="YlGnBu"': Utiliza la paleta de colores "Yellow-Green-
    Blue" para el mapa de calor.
# - 'fmt=".1f"': Formato de los numeros en las anotaciones, en este
     caso no se usan.
sns.heatmap(distance_df, annot=False, cmap="YlGnBu", fmt=".1f")
# Muestra el grafico
plt.show()
```

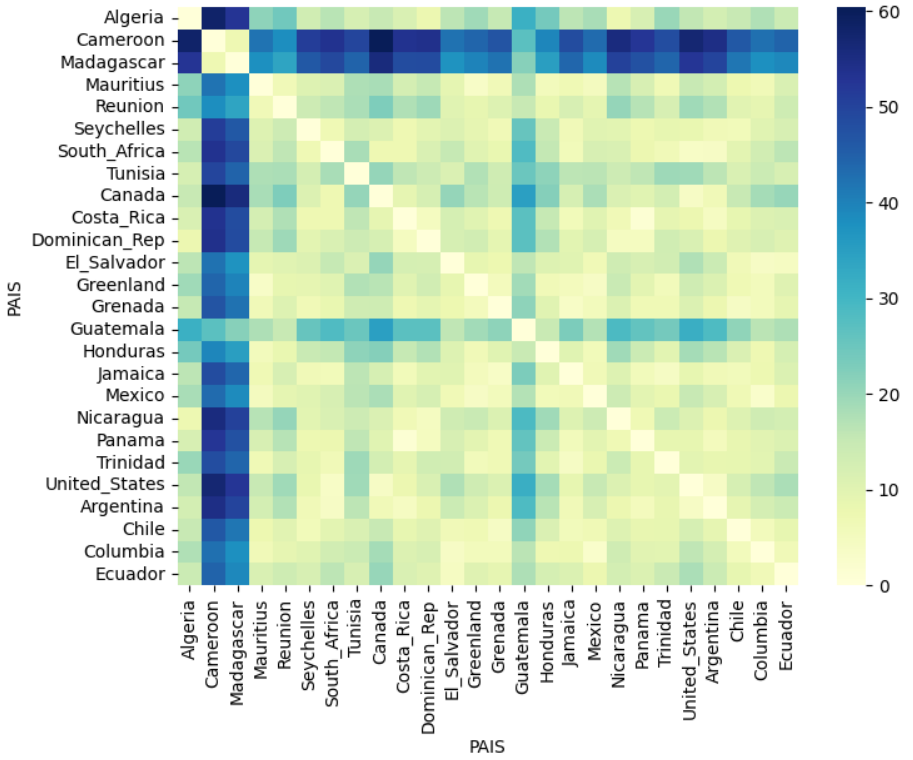


Figura 11: Mapa de calor

Como podemos observar en la Figura 11, hay dos países, Camerún y Madagascar, que tienen una distancia muy superior con el resto de los países.

Una herramienta eficaz para identificar y visualizar patrones entre los diferentes elementos de un conjunto de datos, son los mapas de calor que incorporan técnicas de reordenación basadas en la similitud. Estos métodos nos permiten agrupar los elementos más similares, es decir, aquellos con menor distancia entre sí, mejorando nuestra capacidad para detectar agrupaciones y relaciones clave en los datos.

El gráfico que presentaremos a continuación es un mapa de calor avanzado. Aplicaremos técnicas de clustering jerárquico para reordenar los datos de manera que los elementos similares se ubiquen cercanamente. Esta técnica realza la visualización, facilitando la identificación de grupos y patrones intrínsecos en los datos, crucial para una comprensión profunda y detallada del conjunto de datos analizado.

```
# Realizamos clustering jerarquico para obtener la matriz de enlace
    (linkage matrix).
# Clustermap es una funcion compleja que combina un mapa de calor
    con dendrogramas para mostrar la agrupacion de datos.
# Aqui, estamos usando el dataframe 'distance_df' que contiene las
    distancias euclidianas entre las observaciones.
# La opcion 'cmap' establece la paleta de colores a 'YlGnBu', que
    es un gradiente de azules y verdes.
# La opcion 'fmt' se usa para formatear las anotaciones numericas,
    en este caso a un decimal.
# La opcion 'annot=False' indica que no queremos anotaciones
    numericas en las celdas del mapa de calor.
# La opcion 'method' especifica el metodo de agrupamiento a usar,
    en este caso 'ward'.
linkage = sns.clustermap(distance_df, cmap="YlGnBu", fmt=".1f",
    annot=False, method='ward').dendrogram_row
```

Matriz de Enlace: La matriz de enlace (*linkage*) obtenida del clustering jerárquico contiene información sobre cómo se formaron los *clusters*. Cada fila representa una fusión de dos *clusters* con sus respectivas distancias y el número de elementos en el nuevo *cluster* formado.

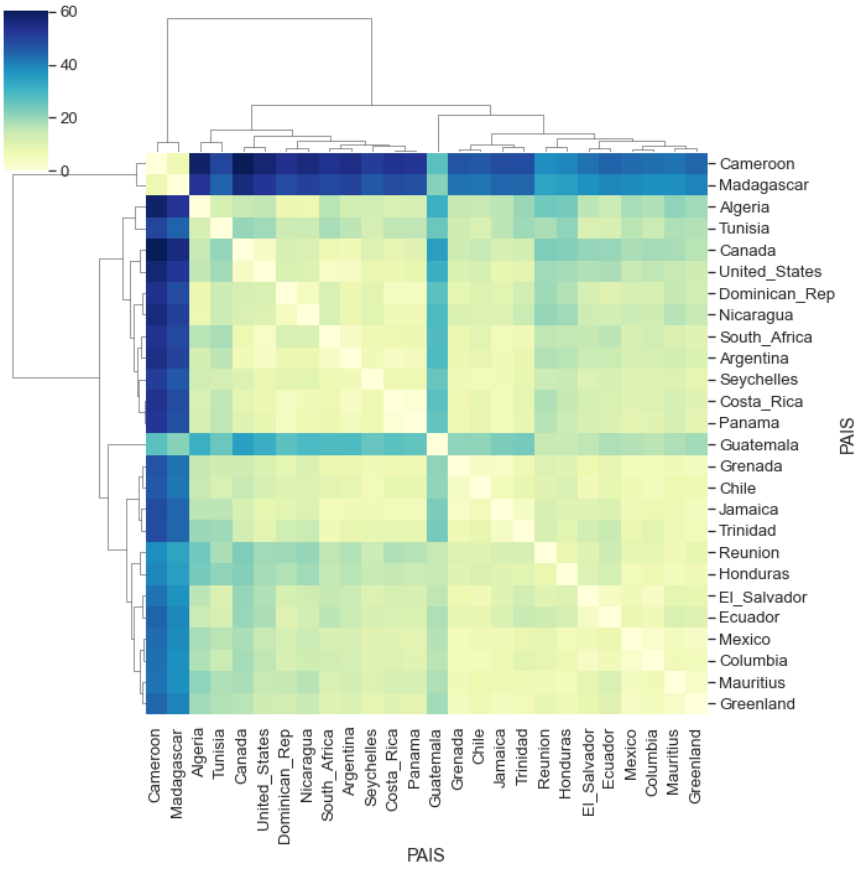


Figura 12: CLustering mapa de calor

Para que la distancia entre individuos no dependa de las unidades en las que están medidas las variables, y refleje mejor las relaciones entre dichos individuos, es conveniente estandarizar las variables. En este ejemplo, todas las variables estan medidas en la misma escala (años) pero no desde el mismo punto de partida, ya que la esperanza de vida se recoge en diferentes etapas. Recordamos que estandarizar una variable es restar su media y dividir por su desviación estándar a cada observación, para conseguir que todas las variables tengan media cero y desviación típica uno:

$$z_{ij} = \frac{x_{ij} - \overline{x}_j}{s_j} \tag{10}$$

```
# Estandarizamos los datos
# Inicializamos el escalador de estandarizacion
scaler = StandardScaler()
```

```
# Ajustamos y transformamos el DataFrame para estandarizar las
    columnas
# 'fit_transform' primero calcula la media y la desviacion estandar
     de cada columna para luego realizar la estandarizacion.
df_std = pd.DataFrame(scaler.fit_transform(df), columns=df.columns)

# Asignamos el indice del DataFrame original 'df' al nuevo
    DataFrame 'df_std'
# Esto es importante para mantener la correspondencia de los
    indices de las filas despues de la estandarizacion.
df_std.index = df.index
```

Ahora, las columnas del conjunto de datos estandarizado tienen una media de 0 y una desviación estándar de 1, lo que se conoce como puntuación z. Esto ayuda a que las variables sean comparables y puede resultar útil para diversas tareas de análisis y modelado de datos.

```
# Calculamos las distancias euclidianas por pares entre las filas
    del DataFrame estandarizado
# 'cdist' calcula la distancia euclidiana entre cada par de filas
    en 'df_std'.
# Esto resulta en una matriz de distancias donde cada elemento [i,
    j] es la distancia entre la fila i y la fila j.
distance_std = distance.cdist(df_std, df_std, 'euclidean')

# Imprimimos los primeros 5x5 elementos de la matriz de distancias
    para tener una vista previa
print(distance_std[:5,:5].round(2))
```

País	Algeria	Cameroon	Madagascar	Mauritius	Reunion
Algeria	0.00	11.6	10.02	6.19	6.32
Cameroon	11.6	0.00	1.87	6.56	6.02
Madagascar	10.02	1.87	0.00	5.5	4.9
Mauritius	6.19	6.56	5.5	0.00	1.39
Reunion	6.32	6.02	4.9	1.39	0.00

Cuadro 2: Matriz de distancia entre países con variables estandarizadas

Podemos ver en la tabla anterior que las distancias son ahora menores, ya que en el proceso de estandarización, en este caso, reduce los valores en las variables. Podemos intentar de nuevo visualizar el mapa de calor de las distancias para datos estandarizados, primero sin ordenar y posteriormente ordenando los países de forma que estén próximos los más parecidos. Conviene preguntarse aquí: ¿cuál es la diferencia respecto al mapa de calor anterior?

```
# Esto determina las dimensiones del grafico
plt.figure(figsize=(10, 8))

# Creamos un nuevo DataFrame para la matriz de distancias
```

```
# 'distance_std' se convierte en un DataFrame con indices y
    columnas correspondientes a 'df_std'
# Esto facilita la interpretacion del mapa de calor, ya que las
    filas y columnas estaran etiquetadas con los indices de 'df_std
    '
df_std_distance = pd.DataFrame(distance_std, index=df_std.index,
    columns=df_std.index)

# Generamos un mapa de calor utilizando Seaborn
# - 'df_std_distance': DataFrame que contiene los datos de
    distancia para visualizar.
# - 'annot=False': No muestra anotaciones numericas en cada celda
    del mapa de calor.
# - 'cmap="YlGnBu"': Define una paleta de colores en tonos de azul
    y verde para el mapa de calor.
# - 'fmt=".1f"': Formato de los numeros en las anotaciones, en este
     caso, un decimal.
sns.heatmap(df_std_distance, annot=False, cmap="YlGnBu", fmt=".1f")

# Mostramos el grafico resultante
plt.show()
```

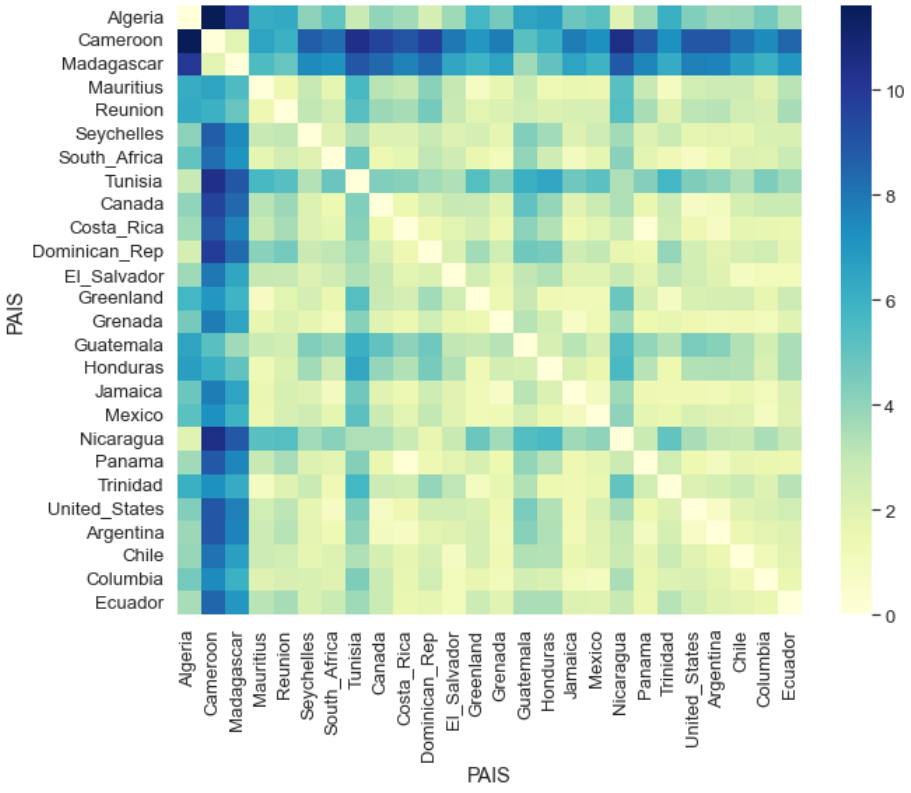


Figura 13: Matriz de distancias estandarizadas entre paises

Volvemos a construir el mapa de calor avanzado donde implementamos un análisis cluster jerárquico, esta vez, sobre la matriz de distancias estandarizadas.

```
# Realizamos clustering jer rquico para obtener la matriz de
    enlace (linkage matrix) sobre las distancias estandarizadas.
linkage = sns.clustermap(df_std_distance, cmap="YlGnBu", fmt=".1f",
     annot=False, method='ward').dendrogram_row
```

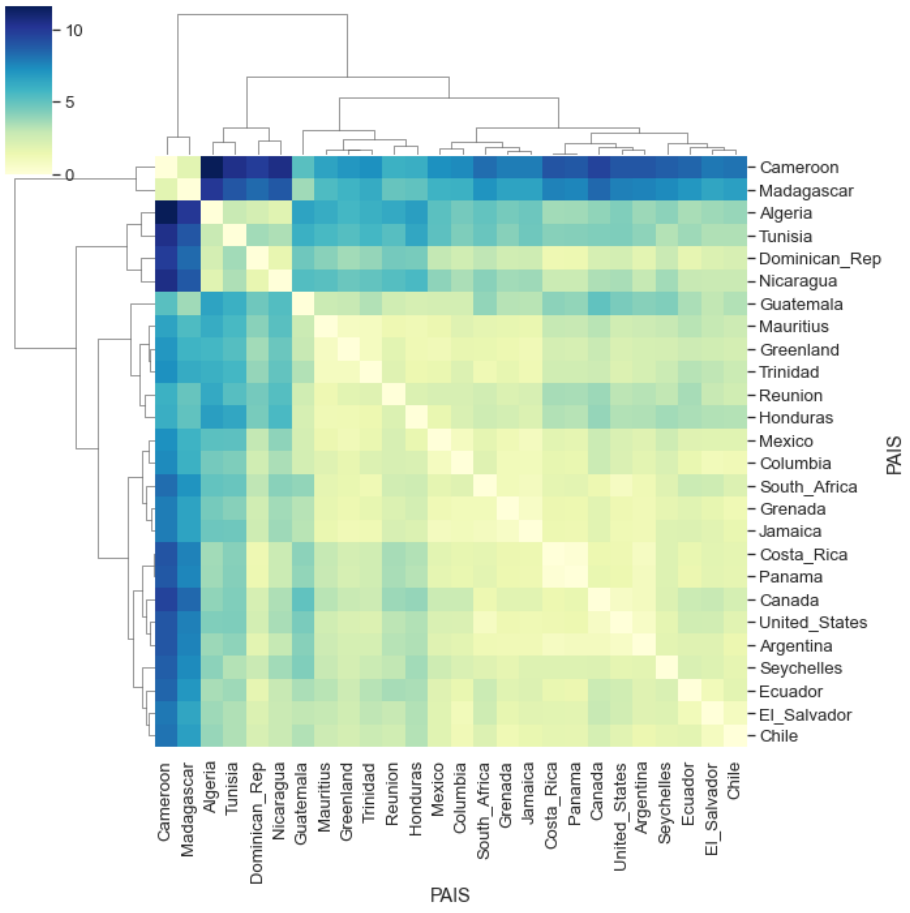


Figura 14: Matriz de distancias visual entre observaciones

Procedemos ahora a la construcción de un dendograma aislado, sin representar el mapa de calor.

```
# Establecemos un umbral de color para el dendrograma

linkage_matrix = sch.linkage(df_std, method='ward')  # Puedes
    elegir un metodo de enlace diferente si es necesario

# Creamos el dendrograma con el umbral de color especificado
dendrogram = sch.dendrogram(linkage_matrix, labels=df_std_distance.
    index.tolist(), leaf_rotation=90)

# Mostramos el dendrograma
plt.show()
```

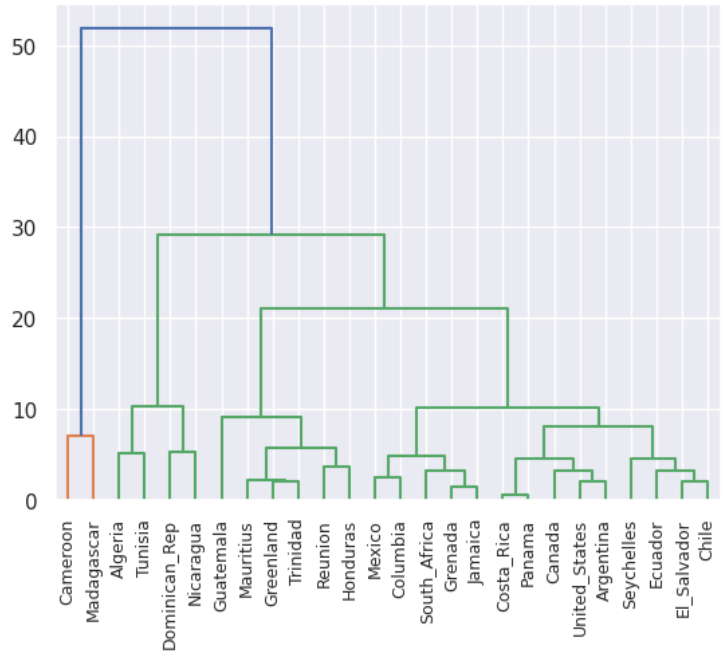


Figura 15: Dendograma

En el dendrograma, los diferentes colores representan distintos conglomerados o grupos identificados por el algoritmo de agrupamiento jerárquico, utilizando el método de Ward. Cada color indica un grupo distinto, de modo que los puntos con el mismo color pertenecen al mismo grupo. La jerarquía del dendrograma se forma al fusionar progresivamente grupos más pequeños en otros más grandes, revelando así la estructura de agrupamiento subyacente.

Para modificar los colores en un dendrograma generado con `scipy.cluster.hierarchy.dendrogram`, puedes utilizar el argumento `color_threshold`. Este argumento determina el valor de umbral para la coloración de los enlaces del dendrograma. Los enlaces por debajo de este umbral se pintarán de acuerdo con sus respectivos clusters, mientras que los enlaces por encima del umbral tendrán el mismo color.

Procedemos con un ejemplo de como se haría:

```
# Establecemos un umbral de color para el dendrograma
color_threshold = 12

# Creamos el dendrograma con el umbral de color especificado
dendrogram = sch.dendrogram(linkage_matrix, labels=df_std_distance.
    index.tolist(), leaf_rotation=90, color_threshold=
    color_threshold)
```

```
# Mostramos el dendrograma
plt.show()
```

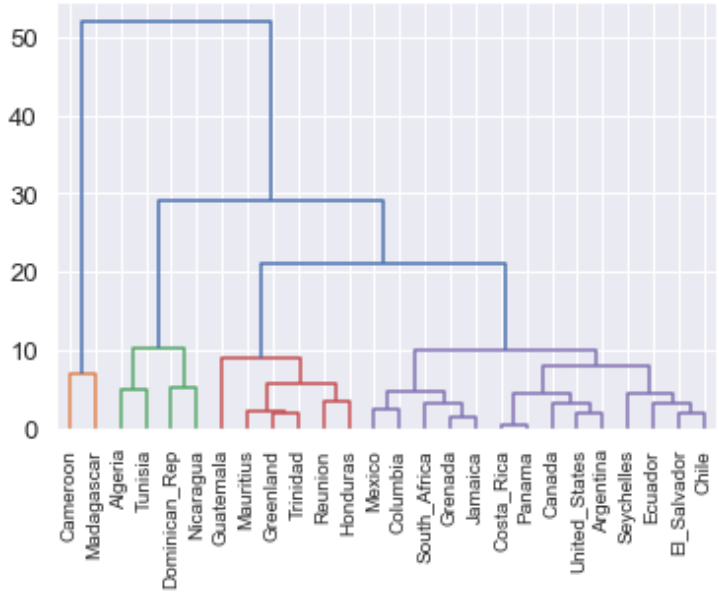


Figura 16: Dendograma umbral de color prefijado

La observación detallada de la estructura del dendrograma nos permite inferir el número más adecuado de clusters. Por ejemplo, en nuestro análisis, un número razonable de clusters podría ser cuatro. Esta conclusión se basa en que las fusiones que forman estos cuatro clusters ocurren a distancias relativamente cortas, indicando una mayor similitud dentro de cada grupo. Por otro lado, para que algunos de estos clusters se combinen entre sí, es necesario considerar distancias mucho mayores, lo que sugiere una diferencia significativa entre estos grupos. Elegir este número de clusters permite una segmentación efectiva del conjunto de datos, facilitando análisis y caracterizaciones posteriores de cada cluster.

Dentro del proceso jerárquico del análisis cluster, podemos prefijar un corte en el dendograma que nos garantice un número máximo de cluster

```
# Asignamos las observaciones de datos a 4 clusters

# Especificamos el numero de clusters a formar
num_clusters = 4

# Asignamos los datos a los clusters
# 'fcluster' forma clusters planos a partir de la matriz de enlace
    'linkage_matrix'
# 'criterion='maxclust'' significa que formaremos un numero maximo
    de 'num_clusters' clusters
cluster_assignments = fcluster(linkage_matrix, num_clusters,
    criterion='maxclust')

# Mostramos las asignaciones de clusters
print("Cluster Assignments:", cluster_assignments)
```

La opción 'maxclust' en el proceso de clustering no jerárquico tiene en cuenta la jerarquía previamente establecida, asegurando que no se formen más clusters que el número especificado ($num_clusters$). Este proceso se logra cortando el dendrograma a una altura determinada, resultando en la formación del número deseado de grupos.

Creamos una nueva variable en nuestro conjunto de datos donde se guarda la información del cluster al que pertenece cada observación.

```
# Creamos una nueva columna 'Cluster4' y asignamos los valores de '
    cluster_assignments' a ella
# Ahora 'df' contiene una nueva columna 'Cluster4' con las
    asignaciones de cluster
df['Cluster4'] = cluster_assignments
```

Para visualizar la distribución espacial de los clusters identificados, una técnica efectiva es la aplicación del Análisis de Componentes Principales (ACP o PCA en inglés). Esta técnica se emplea sobre la matriz de correlaciones de los datos, utilizando principalmente las dos primeras componentes. Esta representación bidimensional es útil siempre que estas componentes capturen una

proporción suficiente de la variabilidad total de los datos.

La implementación de esta técnica puede realizarse fácilmente con herramientas de Python como scikit-learn y matplotlib, siguiendo dos pasos esenciales:

1. Realizar un PCA para reducir la dimensionalidad de los datos a dos componentes principales. Esta reducción facilita la visualización y el análisis, manteniendo al mismo tiempo la mayor parte de la información relevante de los datos.

2. Crear un diagrama de dispersión utilizando las dos componentes principales. En este diagrama, cada punto representa un individuo o una observación, y se colorea según su asignación de cluster. Este tipo de visualización permite observar la agrupación y la separación espacial de los diferentes clusters.

```
# Visualizacion de la distribucion espacial de los clusters
# Paso 1: Realizar PCA
pca = PCA(n_components=2)  # Inicializamos PCA para 2 componentes
    principales
eliminar = ['Cluster4']
principal_components = pca.fit_transform(df.drop(eliminar, axis=1))
      # Transformamos los datos a 2 componentes

fit = pca.fit(df_std)
# Calculamos las dos primeras componentes principales
resultados_pca = pd.DataFrame(fit.transform(df.drop(eliminar, axis
    =1)),
                              columns=['Componente {}'.format(i)
                                  for i in range(1, fit.
                                  n_components_+1)],
                              index=df.index)

# insertamos las componentes principales a la base de datos
    estandarizada.
df_z_cp = pd.concat([df_std, resultados_pca], axis=1)

# Calculo la matriz de correlaciones entre veriables y componentes
Correlaciones_var_comp = df_z_cp.corr()
Correlaciones_var_comp = Correlaciones_var_comp.iloc[:fit.
    n_features_in_, fit.n_features_in_:]



# Creamos un nuevo DataFrame para los componentes principales 2D
# Nos aseguramos de que df_pca tenga el mismo indice que df
df_pca = pd.DataFrame(data=principal_components, columns=['PC1', '
    PC2'], index=df.index)


# Paso 2: Crear un grafico de dispersion con colores para los
    clusters
```

```
plt.figure(figsize=(10, 6))  # Establecemos la dimension del
    grafico

# Recorremos las asignaciones unicas de clusters y trazamos puntos
    de datos con el mismo color
for cluster in np.unique(cluster_assignments):
    cluster_indices = df_pca.loc[cluster_assignments == cluster].
        index
    plt.scatter(df_pca.loc[cluster_indices, 'PC1'],
                df_pca.loc[cluster_indices, 'PC2'],
                label=f'Cluster {cluster}')  # Etiqueta para cada
                    cluster
    # Anotamos cada punto con el nombre del pais
    for i in cluster_indices:
        plt.annotate(i,
                     (df_pca.loc[i, 'PC1'], df_pca.loc[i, 'PC2']),
                         fontsize=10,
                     textcoords="offset points",  # como posicionar
                          el texto
                     xytext=(0,10),  # distancia del texto a los
                         puntos (x,y)
                     ha='center')  # alineacion horizontal puede
                         ser izquierda, derecha o centro

# Lineas de referencia para los ejes x e y
plt.axhline(0, color='black', linestyle='--', linewidth=0.5)
plt.axvline(0, color='black', linestyle='--', linewidth=0.5)

plt.title("Grafico de PCA 2D con Asignaciones de Cluster")
plt.xlabel("Componente Principal 1")
plt.ylabel("Componente Principal 2")
plt.legend()
plt.grid()
plt.show()
```

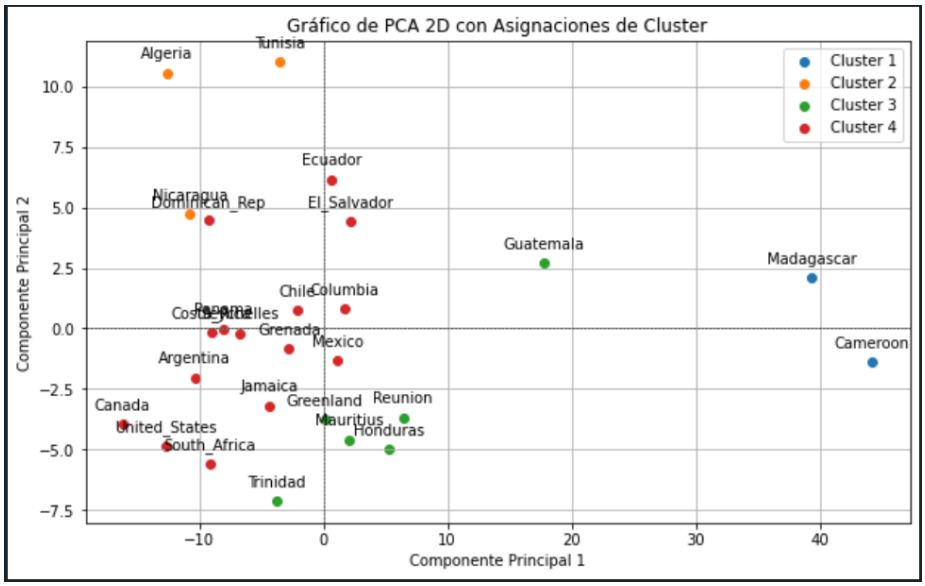


Figura 17: Grafico de PCA 2D con Asignaciones de Cluster

Una vez finalizado el análisis jerárquico, el siguiente paso es determinar el número más adecuado de clusters para nuestro conjunto de datos. Este número se contrastará con la propuesta obtenida del análisis jerárquico, que en nuestro caso sugiere 4 grupos o clusters.

Para este fin, aplicaremos diversos métodos, comenzando con el conocido método del codo:

```
# Metodo del codo
# Creamos un array para almacenar los valores de WCSS para
    diferentes valores de K
wcss = []

for k in range(1, 11):  # Puedes elegir un rango diferente de
    valores de K
    kmeans = KMeans(n_clusters=k, random_state=0)
    kmeans.fit(df_std)
    wcss.append(kmeans.inertia_)  # Inserta es el valor de WCSS

# Graficamos los valores de WCSS frente al numero de grupos (K) y
    buscamos el punto "codo"
plt.figure(figsize=(8, 6))
plt.plot(range(1, 11), wcss, marker='o', linestyle='-', color='b')
plt.title('Metodo del Codo')
plt.xlabel('Numero de Clusters (K)')
plt.ylabel('WCSS')
plt.grid(True)
plt.show()
```

Observando el gráfico del método del codo, identificamos que el WCSS deja de reducirse significativamente a partir de $k = 4$. Esta observación coincide con nuestro análisis previo de clustering jerárquico, reafirmando que un quinto cluster no aportaría una mejora sustancial.

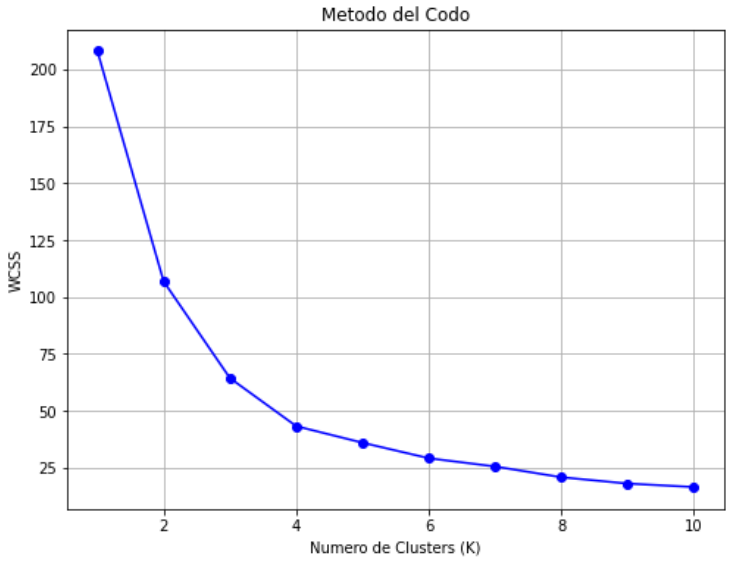


Figura 18: Método del codo

Posteriormente, empleamos el método de la silueta para validar aún más nuestra elección del número óptimo de clusters. Este método evalúa la cohesión y separación de los clusters para diferentes valores de K:

```
# Metodo de la silueta
# Creamos un array para almacenar los puntajes de silueta para
    diferentes valores de K
silhouette_scores = []

# Ejecutamos el clustering K-means para un rango de valores de K y
    calculamos el puntaje de silueta para cada K
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=0)
    kmeans.fit(df_std)
    labels = kmeans.labels_
    silhouette_avg = silhouette_score(df_std, labels)
    silhouette_scores.append(silhouette_avg)

# Graficamos los puntajes de silueta frente al numero de clusters (
    K)
plt.figure(figsize=(8, 6))
plt.plot(range(2, 11), silhouette_scores, marker='o', linestyle='-'
    , color='b')
plt.title('Metodo de la Silueta')
```

```
plt.xlabel('Numero de Clusters (K)')
plt.ylabel('Puntaje de Silueta')
plt.grid(True)
plt.show()
```

El gráfico resultante del método de la silueta nos proporcionará una visualización clara del puntaje de silueta promedio para cada valor de K, lo que nos ayudará a seleccionar el número de clusters que ofrece el mejor equilibrio.

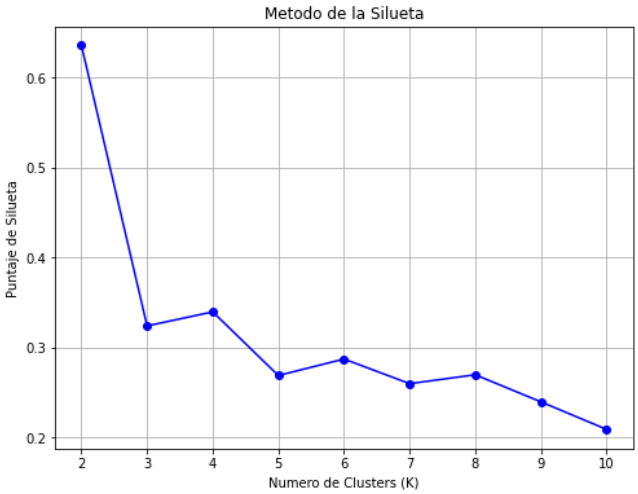


Figura 19: Método del codo

En nuestro caso, el método sugiere $k = 2$ clusters como una solución óptima, pero considerando que esta elección podría ser demasiado simplista, el siguiente máximo local razonable es 4. Esto concuerda con el número de clusters que hemos estado considerando a lo largo de nuestro análisis.

Una vez que hemos determinado el número apropiado de clusters mediante análisis jerárquico y los diferentes métodos utilizados anteriormente, avanzamos hacia la construcción de nuestros grupos o clusters finales a través de un proceso no jerárquico. Este enfoque permite solidificar la estructura de clusters identificada en la etapa jerárquica, proporcionando una segmentación concreta y definida del conjunto de datos.

Utilizamos la función *KMeans()* indicando que el número de cluster es 4. Observamos que, para fijar la generación de valores aleatorios idénticos en todos los ordenadores, necesitamos añadir la siguiente sentencia:

```
# Analisis no jerarquico
# Configurar el numero de clusters (k=4)
k = 4

# Inicializar el modelo KMeans
```

```
# 'n_clusters=k' indica el numero de clusters a formar
# 'random_state=0' asegura la reproducibilidad de los resultados
# 'n_init=10' indica el numero de veces que el algoritmo se
    ejecutara con diferentes centroides iniciales
kmeans = KMeans(n_clusters=k, random_state=0, n_init=10)

# Ajustar el modelo KMeans a los datos estandarizados
# 'df_std' es el DataFrame que contiene los datos previamente
    estandarizados
kmeans.fit(df_std)

# Obtener las etiquetas de los clusters para los datos
# 'kmeans.labels_' contiene la asignacion de cada punto a un
    cluster
kmeans_cluster_labels = kmeans.labels_

# Creamos una nueva columna 'Cluster' y asignamos los valores de '
    kmeans_cluster_labels' a ella
# 'Cluster4_v2' sera el nombre de la nueva columna en el DataFrame
    'df'
df['Cluster4_v2'] = kmeans_cluster_labels

# Ahora 'df' contiene una nueva columna 'Cluster4_v2' con las
    asignaciones de cluster
# Imprimimos los valores de la columna 'Cluster4_v2' para verificar
     las asignaciones de cluster
print(df["Cluster4_v2"])
```

El siguiente paso consiste en calcular y representar el coeficiente de silueta para cada observación dentro de los clusters determinados. El Gráfico de Silueta nos ayudará a evaluar la 'calidad' de cohesión y separación por observación en cada observación en los clusters.

```
# Configuramos el modelo KMeans con 4 clusters y un estado
    aleatorio fijo
kmeans = KMeans(n_clusters=4, random_state=0)
# Ajustamos el modelo KMeans a los datos estandarizados
kmeans.fit(df_std)
# Obtenemos las etiquetas de clusters resultantes
labels = kmeans.labels_

# Calculamos los valores de silueta para cada observacion
silhouette_values = silhouette_samples(df_std, labels)

# Configuramos la dimension de la figura para el grafico
plt.figure(figsize=(8, 6))
y_lower = 10  # Inicio del margen inferior en el grafico

# Iteramos sobre los 4 clusters para calcular los valores de
    silueta y dibujar el grafico
for i in range(4):
    # Extraemos los valores de silueta para las observaciones en el
        cluster i
    ith_cluster_silhouette_values = silhouette_values[labels == i]
    # Ordenamos los valores para que el grafico sea mas claro
    ith_cluster_silhouette_values.sort()
```

```
    # Calculamos donde terminaran las barras de silueta en el eje y
    size_cluster_i = ith_cluster_silhouette_values.shape[0]
    y_upper = y_lower + size_cluster_i

    # Elegimos un color para el cluster
    color = plt.cm.get_cmap("Spectral")(float(i) / 4)
    # Rellenamos el grafico entre un rango en el eje y con los
        valores de silueta
    plt.fill_betweenx(np.arange(y_lower, y_upper),
                      0, ith_cluster_silhouette_values,
                      facecolor=color, edgecolor=color, alpha=0.7)
    # Etiquetamos las barras de silueta con el numero de cluster en
         el eje y
    plt.text(-0.05, y_lower + 0.5 * size_cluster_i, str(i))
    # Actualizamos el margen inferior para el siguiente cluster
    y_lower = y_upper + 10  # 10 para el espacio entre clusters

# Titulos y etiquetas para el grafico
plt.title("Grafico de Silueta para los Clusters")
plt.xlabel("Valores del Coeficiente de Silueta")
plt.ylabel("Etiqueta del Cluster")
plt.grid(True)  # Ponemos una cuadricula para mejor legibilidad
plt.show()  # Mostramos el grafico resultante
```

El Gráfico de silueta resultante muestra la distribución de los coeficientes de silueta para cada clúster. Un coeficiente de silueta más alto sugiere que la observación está bien ajustada en su propio clúster y mal ajustada en clústers vecinos. Observando el gráfico el único cluster que tiene observaciones 'mal' clasificadas es el cluster 1.

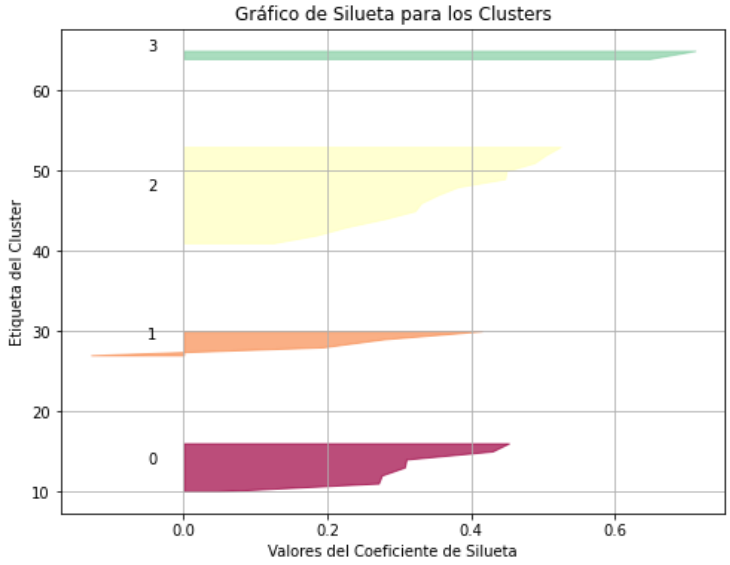


Figura 20: Gráfico de las puntuaciones de silueta de los clústers.

Con los clusters ya definidos, procedemos a visualizar su distribución espacial mediante un Análisis de Componentes Principales (ACP), lo que nos permite observar la agrupación de los datos en un espacio bidimensional.

```
plt.figure(figsize=(10, 6))  # Definir el tama o de la figura

# Iterar a traves de las etiquetas unicas de clusters y graficar
    puntos de datos con el mismo color
for cluster in np.unique(kmeans_cluster_labels):
    cluster_indices = df_pca.loc[kmeans_cluster_labels == cluster].
        index
    plt.scatter(df_pca.loc[cluster_indices, 'PC1'],
                df_pca.loc[cluster_indices, 'PC2'],
                label=f'Cluster {cluster}')  # Poner una etiqueta
                    para cada cluster

    # Anotar cada punto con su respectivo indice
    for i in cluster_indices:
        plt.annotate(i,
                     (df_pca.loc[i, 'PC1'], df_pca.loc[i, 'PC2']),
                         fontsize=10,
                     textcoords="offset points",  # Define como se
                         posicionara el texto
                     xytext=(0,10),  # Define la distancia del
                         texto a los puntos (x,y)
                     ha='center')  # Define la alineacion
                         horizontal del texto
# Lineas de referencia para los ejes x e y
plt.axhline(0, color='black', linestyle='--', linewidth=0.5)
plt.axvline(0, color='black', linestyle='--', linewidth=0.5)
# Configurar el titulo y las etiquetas de los ejes del grafico
plt.title("Grafico 2D de PCA con Asignaciones de Cluster KMeans")
plt.xlabel("Componente Principal 1")
plt.ylabel("Componente Principal 2")
plt.legend()  # Mostrar la leyenda
plt.grid()  # Mostrar la cuadricula
plt.show()  # Mostrar el grafico
```

¿Observas alguna diferencia con respecto a los clusters conseguidos con el método jerárquico?

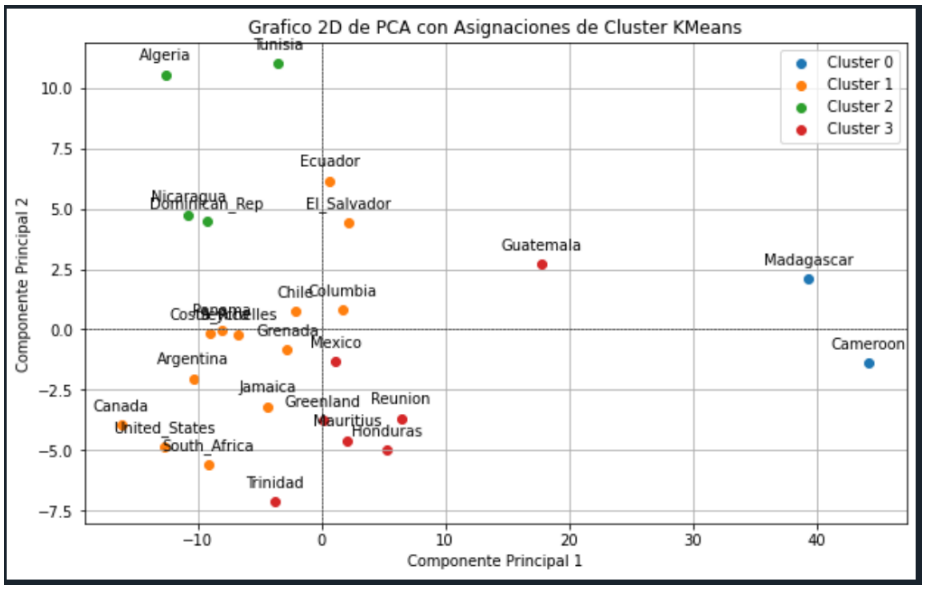


Figura 21: Caption

Para completar el proceso de caracterización, llevamos a cabo un análisis descriptivo de las variables originales dentro de cada cluster, lo que nos permite interpretar y entender las propiedades distintivas de cada grupo.

```
    # Caracterizamos cada cluster
# Asignamos las etiquetas como una nueva columna al DataFrame
    original
df['Cluster'] = labels
# Ordenamos el DataFrame por la columna "Cluster"
df_sort = df.sort_values(by="Cluster")

# Agrupamos los datos por la columna 'Cluster' y calculamos la
    media de cada grupo
# Esto proporcionar  las coordenadas de los centroides de los
    clusters en el espacio de los datos originales
cluster_centroids_orig = df_sort.groupby('Cluster').mean()
# Redondeamos los valores para facilitar la visualizaci n
cluster_centroids_orig.round(2)
# 'cluster_centroids_orig' ahora contiene los centroides de cada
    cluster en el espacio de los datos originales
```

Por ejemplo, los clusters 1 a 2 presentan una mayor esperanza de vida tanto para hombres como para mujeres enm todas las etapas. Es notable que en la etapa inicial y media de la vida, son los clusters 1 y 2 los que exhiben una mayor esperanza, mientras que en la tercera edad, el cluster 1 destaca significativamente. El cluster 3 es el que muestra menor esperanza de vida en cualquier etapa. Este tipo de insights son fundamentales para la interpretación de los resultados y para comprender la naturaleza de los clusters identificados.

Cuadro 3: Las medias (centroides) de los cluster formados

Cluster	$m0$	$m25$	$m50$	$m75$	$w0$	$w25$	$w50$	$w75$
3	36.0	29.5	15.0	6.0	38.0	33.0	18.5	6.5
2	62.46	45.23	24.15	8.54	67.46	49.54	27.23	10.54
1	62.0	48.75	27.5	12.25	66.0	52.5	31.25	14.5
0	58.0	41.86	21.29	6.86	62.0	44.86	24.14	8.29